

Sas POO — Idées de projets

Le dossier `02-sas-poo` est un **module d'entrée**, pas un métier. Il apprend à fabriquer des objets en code : une classe (le moule), des données et des actions. On l'appelle le **sas** parce qu'il relie la semaine 1 à M09, M10, M11 et M13. Un exercice terminé et l'oral informel suffisent pour ouvrir ces quatre métiers.

À lire après `00-OUVREZ-MOI.pdf` et le support projeté. Ce document vous aide à choisir **un exercice** et à commencer. Vous n'avez pas à tous les faire.

Dans chaque exercice, vous fabriquez une chose qui **sait** quelque chose : un fromage prêt, un billet complet, une fiche planète gazeuse ou une planète qui avance. Vous choisissez un exercice à la fois. Le grand projet du module reste facultatif pour ouvrir les quatre métiers.

Planètes (fiches) — simple

Au fond de la salle, quatre cartes : Mercure, Terre, Saturne, Neptune. Ce n'est pas une maquette qui tourne. C'est un **petit catalogue**, comme les fiches d'un musée : un nom, une distance, des satellites, un type. On veut voir quatre **objets**, pas quatre `div` peints à la main.

Quoi mettre en place. Un site `index.html + css/ + js/` (jamais un dossier plat). Une classe `Planete` avec au moins `nom`, `distanceUa`, `satellites`, `type` (rocheuse / gazeuse). Deux méthodes **dans** la classe : `estGazeuse()` et une comparaison (`plusLoinQue(autre)` ou équivalent). Quatre objets affichés : nom, distance, gazeuse oui/non. Zéro canvas. Zéro CSS « ça tourne autour du soleil ».

Univers possibles. Les quatre fiches du cours (Mercure, Terre, Saturne, Neptune). Variante : les mêmes quatre, mais vous changez les nombres (sources notées). Pas une animation d'orbites.

Livable. La page ouverte à 1440 et à 390 + les quatre objets existent + vous expliquez `estGazeuse()` sans lire.

Piège fréquent. Dessiner Saturne qui tourne, puis coller les données dans le HTML. On n'a plus d'objet : on a une illustration. (Les orbites sont **un autre plat**, juste après celui-ci.)

Ce que l'IA n'a pas le droit de remplacer. `estGazeuse()`. Elle peut proposer le squelette de la classe. Vous écrivez cette méthode **une fois à la main**.

Planètes (orbites) — simple → moyen

Le Soleil au centre. Mercure, Terre, Saturne, Neptune autour. Ça bouge. Ce n'est pas le grand projet, et ce n'est pas l'analogie de toute la POO : une classe reste un moule, comme le billet. Ici, l'objet **avance et se dessine**.

Quoi mettre en place. Un site `index.html + css/ + js/` (jamais un dossier plat). Le canvas est **dans** le HTML. Une classe `Planete` : `nom`, `distance`, `rayon`, `couleur`, `vitesse`, `angle`, `type`. Méthodes **dans**

la classe : `avancer()`, `position(cx, cy)`, `dessiner(ctx, cx, cy)`, `estGazeuse()` (même règle qu'aux fiches). Le centre se **passse** : la classe ne le lit pas comme variable globale. La boucle, pour chaque objet : `avancer()` puis `dessiner(ctx, cx, cy)`. Pas de `x / y` calculés à côté. Bouton Pause / Reprendre. Si `prefers-reduced-motion: reduce`, on démarre en pause. Clic (ou focus + Entrée) : nom + gazeuse oui/non.

Univers possibles. Les quatre des fiches, mêmes types. Variante pour les rapides : une vitesse réglable **via la classe**, par exemple `reglerVitesse(v)`. Pas de 5^e planète. Pas huit planètes. Pas de Lune. Pas de ceinture.

Livvable. La page ouverte à 1440 et à 390 + le bouton Pause visible + vous expliquez `avancer()` sans lire.

Piège fréquent. Une fonction `draw()` de deux cents lignes, et des objets vides. On n'a plus d'objet : on a un dessin.

Ce que l'IA n'a pas le droit de remplacer. `avancer()` et `dessiner()`. Elle peut poser le squelette. Vous tapez ces deux méthodes **à la main**.

Données nues puis objet — simple

À la fromagerie du village, on sait si une meule est prête : assez de jours, assez de poids, étiquette écrite. Au début, tout ça tient dans un tableau et trois `if` copiés. Le troisième `if` est déjà faux. On veut le même geste, puis le même geste **dans** l'objet : `meule.estPret()`.

Quoi mettre en place. D'abord un tableau (papier ou Markdown) : trois lignes, colonnes nues, une décision écrite avec des `if`. Ensuite une classe (`Meule` ou `PretMateriel`) et la méthode `estPret()` **dans** la classe. Une petite page ou une console lisible qui affiche les trois cas. Pas des billets de salle : ce plat n'est pas la billetterie.

Univers possibles. Trois meules (tomme jeune, tomme à point, tomme trop tôt). Trois prêts d'école (vidéoprojecteur rendu, câble HDMI encore dehors, enceinte réservée demain). Trois pains d'une boulangerie (même règle, autre nom).

Livvable. Le tableau nu **et** les trois objets. Vous montrez le `if` d'avant, puis vous dites ce que `estPret()` décide, sans lire.

Piège fréquent. Garder le `if` dans la page et laisser la classe vide. Ce n'est plus un objet : c'est une étiquette.

Ce que l'IA n'a pas le droit de remplacer. Le passage du tableau à la méthode. Elle ne saute pas l'étape nue. Elle n'écrit pas `estPret()` à votre place la première fois.

Trois produits — simple

Le food-truck du quai a trois choses ce soir : féra, frites, soupe. Chacune a un nom, un prix, un stock. On veut le **même geste** que le plat fromage, dans un autre domaine, pour que « objet » ne reste pas

collé à une seule histoire.

Quoi mettre en place. Une classe `Produit` (ou `Plat`) : nom, prix, stock. Une méthode dans la classe (`estDispo()`, `prixAffiche()`, au choix — **une** règle métier, tenue). Trois objets affichés sur une petite page (`index.html + css/ + js/`). Pas de panier, pas de paiement, pas de framework.

Univers possibles. Camion du quai (fêra, frites, soupe). Stand de gymnase (barres, oranges, eau). Comptoir de buvette de club (thé, gâteau, soupe).

Livable. Trois objets, une méthode dans la classe, page lisible à 390. Vous expliquez la méthode sans lire.

Piège fréquent. Trois `const` et un `if` dans `index.html`. Le plat demande le même geste **objet** que le fromage, pas une liste décorée.

Ce que l'IA n'a pas le droit de remplacer. La règle métier (quand c'est dispo, quand le prix change). Vous l'écrivez à la main une fois.

Commande de 3 billets — moyen

À la caisse de la salle, personne n'achète « un titre » tout seul. On achète **une commande** : trois billets, un total. Le billet sait s'il est complet. La commande sait additionner. Deux objets, l'un contient l'autre.

Quoi mettre en place. Une classe `Billet` (titre, prix, places). Une classe `Commande` qui **tient** trois billets (tableau d'objets, pas trois variables perdues). Une méthode de total **dans** `Commande`. Page : les trois lignes + le total. Arbre `index.html + css/ + js/`.

Univers possibles. Salle de village (trois soirées). Club de jazz (trois dates). Ciné-club de l'école (trois séances). Watt n'est pas obligatoire.

Livable. La page avec un total qui vient de la commande, pas d'un `+` écrit à la main dans le HTML. Vous dites, sans lire, qui additionne.

Piège fréquent. Calculer le total dans la page, et laisser `Commande` vide. Ce n'est plus de la composition : c'est un titre.

Ce que l'IA n'a pas le droit de remplacer. Le total dans `Commande`. Elle peut poser les deux classes. Vous écrivez l'addition.

Même objet, deux langages — moyen

L'idée ne doit pas mourir en changeant de fichier. Un `Billet` en JavaScript, le même `Billet` en PHP : mêmes propriétés, mêmes méthodes, autre syntaxe. M09 et M10 parlent PHP dès le premier jour. Si l'objet ne tient qu'en JS, M09 et M10 commenceront dans le brouillard.

Quoi mettre en place. Semaine possible 1 : classe `Billet` en JS, trois objets (complet / presque / vide), page lisible. Semaine possible 2 : **la même** classe en PHP, page `billets.php`, sans base de données. Vous dites à voix haute ce qui n'a **pas** changé. L'énoncé tient les deux langages : vous

pouvez les étaler sur ~2 semaines.

Univers possibles. Billets de salle (Paléo, club, salle des fêtes). Variante : trois ateliers d'une asso, même objet, autre nom. Pas le système solaire.

Livable. Page JS + page PHP + une minute : « voilà ce qui n'a pas bougé ».

Piège fréquent. Recopier le HTML et oublier les méthodes. Ou ouvrir PDO « tant qu'on y est » : c'est M10.

Ce que l'IA n'a pas le droit de remplacer. `estComplet()` écrite à la main **une fois** (JS ou PHP). La comparaison orale est à vous.

Catalogue affiché — avancé

Un festival a plus de trois lignes. On veut une **liste d'objets** qui fabrique la page : chaque carte vient d'un objet, pas d'un copier-coller HTML. C'est le pont vers M10 (annuaire) et M11 (l'écran suit les données).

Quoi mettre en place. Une classe du domaine (`Concert`, `Stand`, `Atelier...`). Au moins quatre objets. Une page `index.html` + `css/` + `js/` qui **parcourt** le tableau et dessine les cartes. Une méthode dans la classe (`complet`, `gratuit`, `tout public...`). Mobile : ça se lit à 390.

Univers possibles. Festival de rive (quatre concerts). Marché de producteurs (quatre stands). Semaine des métiers à l'école (quatre ateliers). Pas Watt recopié, pas les quatre planètes du plat 01.

Livable. La page générée depuis les objets + vous montrez la boucle + vous expliquez la méthode.

Piège fréquent. Quatre articles écrits à la main, et un tableau « pour plus tard ». Si on change un prix dans l'objet et que la carte ne suit pas, ce n'est pas encore ce plat.

Ce que l'IA n'a pas le droit de remplacer. La boucle qui affiche, et la méthode métier. Elle ne « fait pas le site du festival ».

JSON aller-retour — avancé

Le prêt de matériel de l'école vit dans un fichier. Le matin on le lit, le soir on l'écrit. Les objets ne sont plus seulement en mémoire : ils **vont** et ils **reviennent**. Sans ça, M10 commencera dans le brouillard.

Quoi mettre en place. Un fichier JSON (trois à cinq matériels : nom, dispo, qui, date). Une classe `Matériel` (ou `Pret`) avec une méthode (`estDispo()`, `estEnRetard()`). Lire le fichier, créer les objets, afficher. Modifier **un** état, **réécrire** le fichier. JS (fetch + download, ou Node si le prof l'autorise) **ou** PHP `file_get_contents` / `file_put_contents`. Pas de MySQL.

Univers possibles. Prêt école (vidéoprojecteur, câble, enceinte, caméra). Réservation de salles de l'établissement. Casiers d'instruments d'une fanfare.

Livable. Le JSON avant / après + la page ou le script + vous dites où vit la règle, et où vit le fichier.

Piège fréquent. Tout recalculer à la main dans le JSON, et laisser la classe décorative. Ou ouvrir une

base : hors sujet.

Ce que l'IA n'a pas le droit de remplacer. La règle dans la classe, et la preuve que le fichier a changé **parce que** vous êtes passé par l'objet.

Grand projet — mini-domaine montrable

Ce projet permet de montrer un petit domaine **à vous**, mais il n'est pas obligatoire pour ouvrir M09, M10, M11 ou M13. Il ne reprend pas les fiches planètes. Vous réalisez une classe, un affichage et **une** règle métier que vous expliquez sans lire. Un camarade ouvre la page et comprend de quoi il s'agit.

Quoi mettre en place. Un univers **nouveau** (fromagerie, food-truck, asso, ludothèque, spots — au choix, **pas** le système solaire). Une classe. Une page `index.html` + `css/` + `js/` (ou PHP si vous sortez du plat deux-langages). Une règle dans la classe. Oral informel : l'objet, la méthode, pourquoi ce n'est pas une liste de variables.

Univers possibles. Emprunts de la médiathèque. Commandes d'un camion. Inscriptions à un atelier. Inventaire d'une fanfare. Tout sauf refaire les quatre planètes.

Livrable. La page + la classe + deux minutes sans lire.

Piège fréquent. Livrer une animation, un framework, ou « j'ai fait le système solaire ». Ce plat demande un domaine montrable, une règle, un oral.

Ce que l'IA n'a pas le droit de remplacer. Le domaine (le vôtre), la règle métier et l'oral informel. Vous devez pouvoir expliquer vous-même la méthode choisie.

Projet libre — à convenir

Le grand projet ne suffit pas, ou il ne convient pas. Ici : un brief d'**une page**, validé **avant** de produire. Le sas ne change pas : une classe, des objets, une méthode dans la classe, un affichage.

Quoi mettre en place. `sas/brief-libre.md` : pour qui, livrable, langage (JS et/ou PHP), les **non** (pas d'orbites comme but, pas de base, pas de framework imposé). Visa du prof. Puis seulement le code.

Univers possibles. À écrire avec le prof. Même exigence que le plat 08 : pas le système solaire comme grand soir.

Livrable. Ce que le brief a listé. L'oral est le même.

Piège fréquent. Commencer le code pour « voir », puis coller le brief. Sans visa, ce n'est pas ce plat.

Ce que l'IA n'a pas le droit de remplacer. Le brief. Elle implémente *ceci* après le visa.